ParkIQ Parking Data Model

A schema-as-configuration geodatabase design for parking site selection: one YAML file builds, checks and documents every layer, domain and relationship, for any market

Feature classes	**31**	Coded and range domains	**14**
Attribute tables	**13**	Relationship classes	**11**
Raw source snapshots	**19**	Fields defined	**369**
Live sources loaded (Franklin County, OH)	**20**	Parcels in the run	**492,935**
QA checks logged	**115**	Schema diff, live run vs definition	**0 differences**

Numbers are generated from run `20260925_2217_franklin_oh_Balanced` and schema v1.0.0 when this report is built; none are typed by hand.

The Problem

A parking site-selection model pulls together more than a dozen public datasets (parcels, zoning, jobs, census block groups, transit schedules, flood zones, building footprints, parking lots, meters) and produces many derived layers: a hexagon grid, demand and supply by time of day, hot zones, candidate parcels, scores and pro formas. Without a deliberate data model, that turns into a folder of one-off files where:

- field names, units and allowed values drift between steps, so a join or a filter silently drops records;
- nobody can say which source and vintage a number came from, or which run produced it;
- the design lives in someone's head, so a new market or a new analyst starts from scratch;
- the documentation (ERD, data dictionary) is drawn once by hand and is wrong by the next change.

My Approach

The schema is configuration. Every feature class, table, field, type, domain, subtype and relationship is declared once in `schema/schema.yaml`. The same file drives four things, so they can never disagree:

- **BuildSchema** creates every layer and table, empty, in a new GeoPackage, and writes the domains as GeoPackage schema-extension constraints so GIS software sees them as field domains.
- **Validation on write**: every time a pipeline step writes a layer, the data is coerced and checked against the schema (types, nullability, uniqueness, domain values); a violation stops the run with every problem listed.
- **schema-diff** compares any GeoPackage with the definition and lists every difference: missing layers, wrong geometry or CRS, missing or extra columns, detached domains.
- **Documentation**: the ERD (draw.io and PNG) and the data dictionary are generated from the file; a test fails if the committed documents are out of date.

One schema for every market. Markets differ only in their configuration file (boundary, projection, source settings), never in the data model, so a second city reuses the design unchanged.

Lineage and provenance built in. Every feature class carries `source_id`, `run_id` and `load_ts`. Each run writes one GeoPackage plus a source registry (endpoint, vintage, download date, licence, native CRS, transformation, row count, status) and a QA log, so any number traces back to a source and a run.

Technical Highlights

- **Feature datasets mirror the analysis**: Reference (boundary, study area, H3 grid, walk network, zoning, flood, environmental sites), Cadastral (parcels, buildings), Demand (anchors, transit, venues, jobs, block groups), Supply (facilities, metered curb), Analysis (hot zones, candidate parcels, walk sheds) and Results (scores, financials, shortlist).
- **Domains enforce vocabulary**: land-use class, owner type, zoning screen, anchor category, supply type, daypart, screen status, scenario and tenure are coded domains; scores (0–100) and occupancy (0–1) are range domains. The constraints are re-attached after every write, because GDAL drops them when it replaces a table.
- **Subtypes and relationship classes**: parcels are subtyped by land-use class and supply facilities by type; eleven relationship classes (for example candidate parcel → scores, run → scores, hex → demand by daypart) are recorded in the GeoPackage and become real relationship classes in the file-geodatabase export.
- **Hex × time-of-day results as tables**: demand, supply and gap per hex and daypart are attribute tables keyed by the H3 cell id and joined to the 16,069-cell grid for mapping, instead of storing each hexagon five times.
- **Immutable raw snapshots**: each source is also kept as delivered, in WGS 84, beside its standardized layer.
- **Fast, reproducible builds**: an empty template per schema version and CRS is built once and copied into each new run.

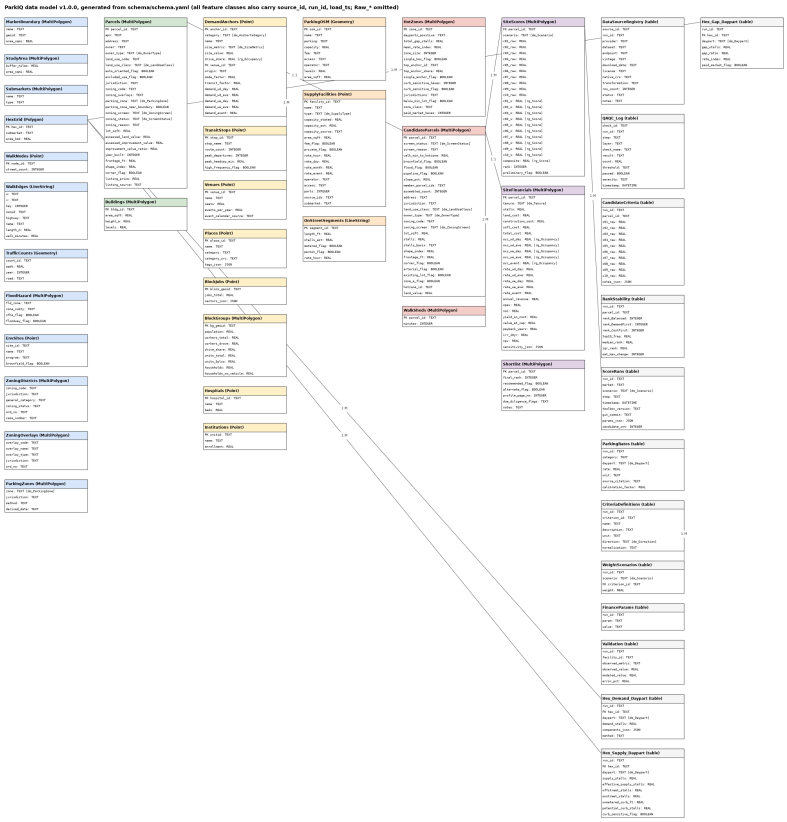

Figure 1. Entity-relationship diagram, generated from `schema.yaml` (v1.0.0). Colours are feature datasets; lines are relationship classes; PK/FK mark keys.

Results

- A live run for Franklin County, Ohio, built all layers from the definition and wrote 59 of them with data; `schema-diff` reports **0 differences** between the run and the definition.
- 20 sources loaded with a registry row each (5 licensed or pending sources are recorded as "not configured", never substituted with invented data); 115 QA checks are logged, with 0 error-level failures.
- The same parcels layer (492,935 parcels) carries land use, owner type, zoning code, overlays, downtown parking zone and the zoning-screen result, all checked against their domains on every write (Figure 2).

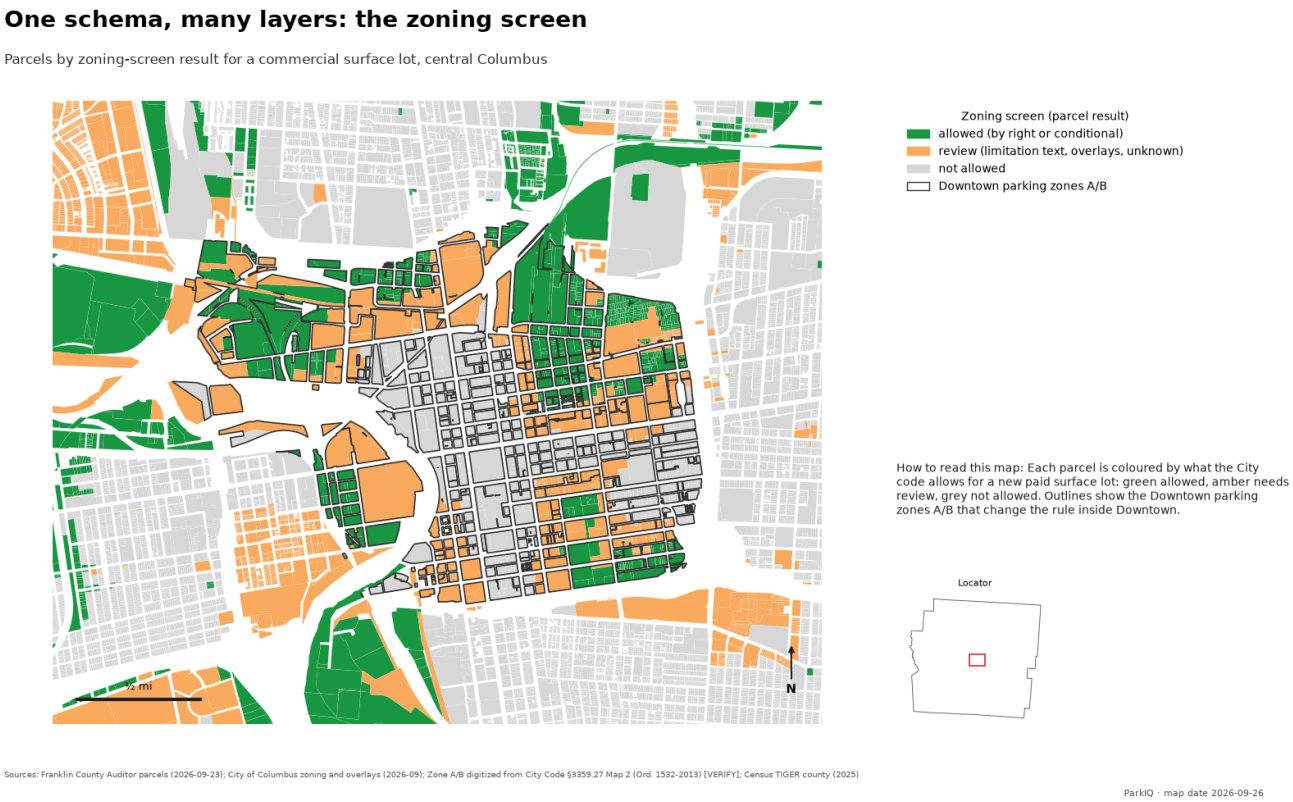

Figure 2. The zoning-screen result stored on each parcel, central Columbus. Preliminary; no parcel identifiers are shown.

Limitations and next steps

- Relationship classes and subtypes are recorded in the GeoPackage but only become enforced objects in the file-geodatabase export (next step), because GeoPackage has no relationship class that ArcGIS reads.
- Topology rules (parcels must not overlap; the hex grid has no gaps) are defined but not yet checked in the automated QA.
- Downtown parking zones A/B are digitized from a map in the City code until the City confirms whether an official GIS layer exists.

Data sources

Source	Dataset	Vintage	Licence
US Census Bureau	TIGER/Line Counties (boundary)	2025	Public domain
County assessor / auditor (Regrid fallback)	Parcel polygons + CAMA attributes	2026-09-23	per county portal
Overture Maps Foundation	Buildings theme	2026-09-23.1	ODbL (to confirm)
OpenStreetMap contributors	POIs, food, retail, lodging, venues, health, education (via OSMnx / Overpass)	nan	ODbL
US Census Bureau LEHD	LODES WAC (all jobs JT00, all workers S000) + geography crosswalk	2023	Public domain (to confirm)

Source	Dataset	Vintage	Licence
US Census Bureau	ACS 5-year block groups (B01003, B08301, B25024, B25044) + TIGER block groups	2024	Public domain (to confirm)
OpenStreetMap contributors	amenity=parking (via OSMnx / Overpass)	nan	ODbL
OpenStreetMap contributors	Pedestrian network (OSMnx network_type=walk)	nan	ODbL
Transit agency / Mobility Database	GTFS schedule	2026-09-07	per agency (to confirm)
CMS Provider of Services (Hospital & other), geocoded	Hospitals with bed counts	2026-Q2	Public domain (CMS)
NCES	IPEDS directory (HD) + 12-month enrollment (EFFY)	2024	Public domain (to confirm)
ODOT Office of Technical Services (TIMS)	AADT traffic counts	2025	Public web service [VERIFY terms] (to confirm)
FEMA	National Flood Hazard Layer, flood hazard zones	2026-09	Public domain (to confirm)
US EPA	FRS facilities / ACRES brownfields	2026-09	Public domain (to confirm)
USGS	3DEP 1/3 arc-second DEM	2026-08	Public domain (to confirm)
City of Columbus	On-street curb inventory by block face, metered spaces and posted rates	2026-09	Open data [VERIFY licence: sibling City items are CC0; this layer has no Hub item] (to confirm)
City of Columbus	Base Zoning Districts (Title 33 + Title 34 codes; L- prefix fused, e.g. LC4)	2026-09	Open data, 'AS IS'
City of Columbus	Commercial Zoning Overlays (UCO/CCO/RCO) and Planning Overlays (incl. University NC/RC)	2026-09	Open data, 'AS IS'
City of Columbus	Downtown District parking zones A/B (§3359.27, Map 2), DERIVED	Ord. 1532-2013 (Map 2 image dated 2026-07-01)	City code (public record)
AASHTO CTPP / US Census Bureau	CTPP 2017-2021 table B202105 Means of Transportation to Work (workplace geography), tract of work	2017-2021	Public (CTPP data portal; registered API key)

Why It Matters

Site-selection and investment decisions only hold up if every number can be traced to its source and recomputed. Treating the data model as configuration makes the geodatabase self-describing and self-checking: the build, the validation and the documentation come from one definition, a new market is a new configuration file rather than a new project, and a reviewer can see exactly which data, vintage and run produced a result.

Repository: https://github.com/ndeogobernard/parkiq-location-intelligence · Portfolio: https://ndeogobernard.github.io/ndeogo/ · Report generated 2026-09-26